

LAPORAN RESMI

“Enkapsulasi”



Disusun oleh:

Nama: Kristian Utama Putra

NRP: 3125600047

Kelas: 2 STr. Teknik Informatika B

**PROGRAM STUDI SARJANA TERAPAN TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA**

2026

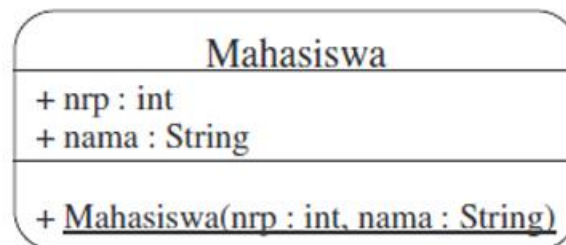
TUGAS PENDAHULUAN

1. Apakah yang dimaksud dengan enkapsulasi?
Encapsulation (Enkapsulasi) adalah suatu cara untuk menyembunyikan implementasi detail dari suatu class.
2. Apakah yang dimaksud dengan constructor?
Constructor (konstruktor) adalah suatu method yang pertama kali dijalankan pada saat pembuatan suatu obyek.
3. Apakah yang dimaksud dengan overloading constructor?
Terdapat lebih dari satu constructor dalam suatu class.

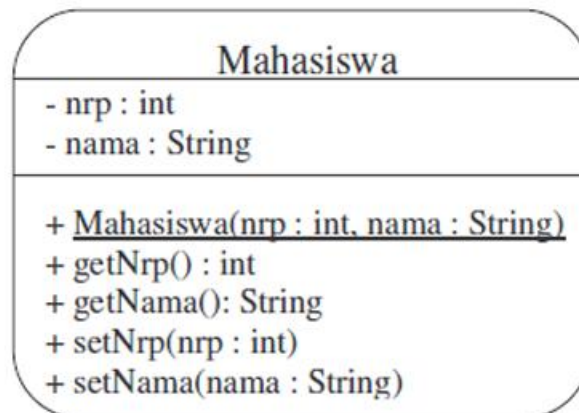
PERCOBAAN

Percobaan 1 : Melakukan enkapsulasi pada suatu class

Implementasikan UML class diagram Mahasiswa sebelum dan setelah dilakukan enkapsulasi!



Jika enkapsulasi dilakukan pada class diagram diatas, maka akan berubah menjadi:



SC:

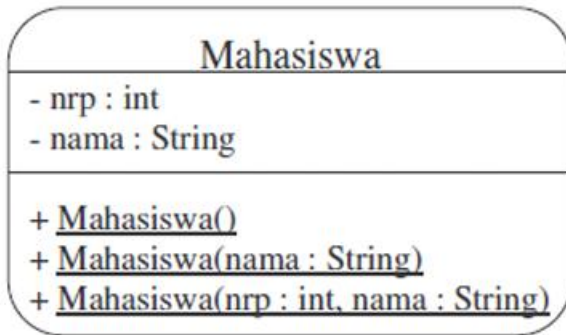
Sebelum:

```
public class Mahasiswa {  
    private int nrp;  
    private String nama;  
    public Mahasiswa() {  
        nrp=0;  
        nama="";  
    }  
  
    public Mahasiswa(String nama) {  
        nrp=0;  
        this.nama=nama;  
    }  
  
    public Mahasiswa(int nrp, String nama) {  
        this.nrp=nrp;  
        this.nama=nama;  
    }  
}
```

Sesudah:

```
public class Mahasiswa {  
    private int nrp;  
    private String nama;  
  
    public Mahasiswa(int nrp, String nama) {  
        this.nrp = nrp;  
        this.nama = nama;  
    }  
  
    public int getNrp() {  
        return nrp;  
    }  
  
    public String getNama() {  
        return nama;  
    }  
  
    public void setNrp(int nrp) {  
        this.nrp = nrp;  
    }  
  
    public void setNama(String nama) {  
        this.nama = nama;  
    }  
  
    public static void main(String[] args) {  
        Mahasiswa mhs = new Mahasiswa(123, "Budi");  
  
        System.out.println("NRP : " + mhs.getNrp());  
        System.out.println("Nama : " + mhs.getNama());  
  
        mhs.setNrp(456);  
        mhs.setNama("Andi");  
        System.out.println("NRP : " + mhs.getNrp());  
        System.out.println("Nama : " + mhs.getNama());  
    }  
}
```

Percobaan 2 : Melakukan overloading constructor



Dari class diagram tersebut, dapat diimplementasikan ke dalam program sebagai berikut:

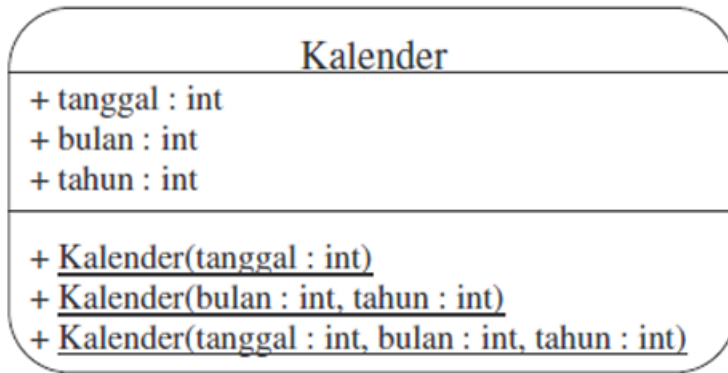
```
public class Mahasiswa {
    private int nrp;
    private String nama;
    public Mahasiswa() {
        nrp=0;
        nama="";
    }

    public Mahasiswa(String nama) {
        nrp=0;
        this.nama=nama;
    }

    public Mahasiswa(int nrp, String nama) {
        this.nrp=nrp;
        this.nama=nama;
    }
}
```

LATIHAN

Latihan 1: Mengimplementasikan UML class diagram dalam program untuk class Kalender



Dari class diagram diatas, desainlah suatu class yang memenuhi konsep enkapsulasi. Untuk nilai inisialisasi, dipakai 1-1-2000. Pakailah kata kunci `this` untuk mempersingkat pengkodean.

SC:

```
public class Kalender {
    private int tanggal;
    private int bulan;
    private int tahun;

    public Kalender(int tanggal, int bulan, int tahun) {
        this.tanggal = tanggal;
        this.bulan = bulan;
        this.tahun = tahun;
    }

    public Kalender(int tanggal) {
        this(tanggal, 1, 2000);
    }

    public Kalender(int bulan, int tahun) {
        this(1, bulan, tahun);
    }

    public int getTanggal() {
        return tanggal;
    }

    public int getBulan() {
        return bulan;
    }
}
```

```

public int getTahun() {
    return tahun;
}

public void setTanggal(int tanggal) {
    this.tanggal = tanggal;
}

public void setBulan(int bulan) {
    this.bulan = bulan;
}

public void setTahun(int tahun) {
    this.tahun = tahun;
}

public void tampil() {
    System.out.println(tanggal + "-" + bulan + "-" + tahun);
}

public static void main(String[] args) {
    new Kalender(17).tampil();
    new Kalender(8, 2015).tampil();
    new Kalender(17, 8, 1945).tampil();
}
}

```

Tulislah listing program berikut ini sebagai pengetesan

```

public class TesKalender {
    public static String getTime(Kalender kal) {
        String tmp;
        tmp = kal.getTanggal() + "-" +
            kal.getBulan() + "-" +
            kal.getTahun();
        return tmp;
    }
    public static void main(String args[]) {
        Kalender kal = new Kalender(8);
        System.out.println("Waktu awal : " + getTime(kal));
        kal.setTanggal(9);
        System.out.println("1 hari setelah waktu awal : "+getTime(kal));
        kal = new Kalender(6, 2003);
        System.out.println("Waktu berubah : " + getTime(kal));
        kal.setBulan(7);
    }
}

```

```

        System.out.println("1 bulan setelah itu : " + getTime(kal));
        kal = new Kalender(20, 10, 2004);
        System.out.println("Waktu berubah : " + getTime(kal));
        kal.setTahun(2005);
        System.out.println("1 tahun setelah itu : " + getTime(kal));
    }
}

```

Output yang diinginkan:

```

Waktu awal : 8-1-2000
1 hari setelah waktu awal : 9-1-2000
Waktu berubah : 1-6-2003
1 bulan setelah itu : 1-7-2003
Waktu berubah : 20-10-2004
1 tahun setelah itu : 20-10-2005

```

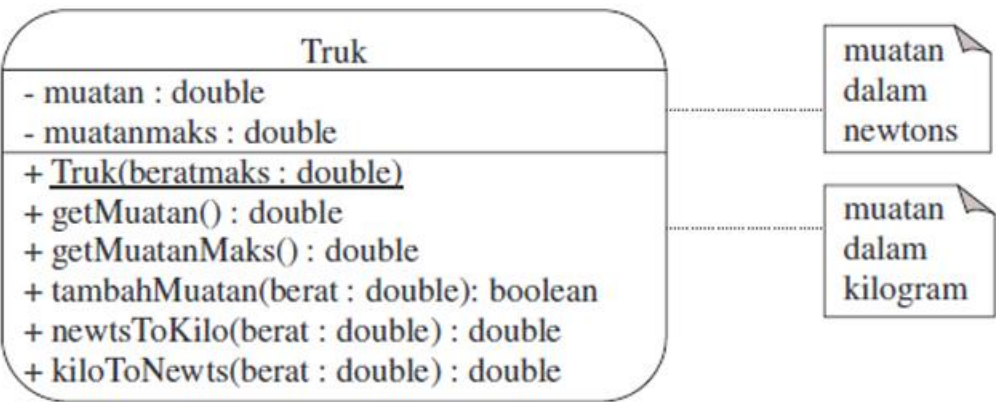
Output:

```

Waktu awal : 8-1-2000
1 hari setelah waktu awal : 9-1-2000
Waktu berubah : 1-6-2003
1 bulan setelah itu : 1-7-2003
Waktu berubah : 20-10-2004
1 tahun setelah itu : 20-10-2005

```

Latihan 2 : Mengimplementasikan UML class diagram dalam program untuk class Truk



Keterangan : 1 kilogram = 9,8 newtons

Transformasikan class diagram diatas ke dalam bentuk program!

SC:

```

public class Truk {
    private double muatan;

```

```
private double muatanmaks;

public Truk(double beratmaks) {
    this.muatan = 0;
    this.muatanmaks = kiloToNewts(beratmaks);
}

public double getMuatan() {
    return newtsToKilo(muatan);
}

public double getMuatanMaks() {
    return Math.round(newtsToKilo(muatanmaks) * 1000) / 1000.0;
}

public boolean tambahMuatan(double berat) {
    double beratNewtons = kiloToNewts(berat);
    if (muatan + beratNewtons <= muatanmaks) {
        muatan += beratNewtons;
        return true;
    }
    return false;
}

public double newtsToKilo(double berat) {
    return berat / 9.8;
}

public double kiloToNewts(double berat) {
    return berat * 9.8;
}

public static void main(String[] args) {
    Truk truk = new Truk(1000);

    System.out.println("Muatan maks : " + truk.getMuatanMaks() + " kg");
    System.out.println("Tambah 600 kg : " + truk.tambahMuatan(600));
    System.out.println("Tambah 300 kg : " + truk.tambahMuatan(300));
    System.out.println("Tambah 200 kg : " + truk.tambahMuatan(200));
    System.out.println("Muatan sekarang : " + truk.getMuatan() + " kg");
}
}
```

Tulislah listing program berikut ini sebagai pengetesan.

```
public class TesTugas2 {
    public static void main(String args[]){
        boolean status;
        Truk truk = new Truk(900);
        System.out.println("Muatan maksimal = "+truk.getMuatanMaks());
        status = truk.tambahMuatan(500.0);
        System.out.println("Tambah muatan : 500");
        if (status)
            System.out.println("Ok");
        else
            System.out.println("Gagal");

        status = truk.tambahMuatan(300.0);
        System.out.println("Tambah muatan : 300");
        if (status)
            System.out.println("Ok");
        else
            System.out.println("Gagal");

        status = truk.tambahMuatan(150.0);
        System.out.println("Tambah muatan : 150");
        if (status)
            System.out.println("Ok");
        else
            System.out.println("Gagal");

        status = truk.tambahMuatan(50.0);
        System.out.println("Tambah muatan : 50");
        if (status)
            System.out.println("Ok");
        else
            System.out.println("Gagal");
        System.out.println("Muatan sekarang = " + truk.getMuatan());
    }
}
```

Output yang diinginkan:

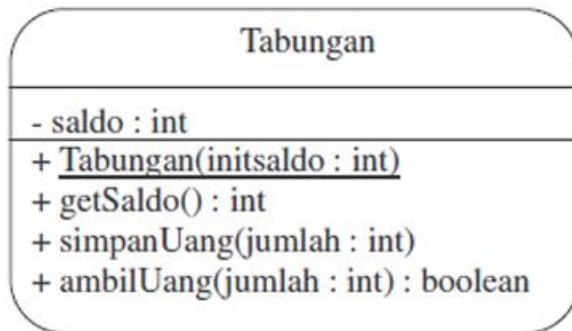
```
Muatan maksimal : 900.0
Tambah muatan : 500 ok
Tambah muatan : 300 ok
Tambah muatan : 150 gagal
Tambah muatan : 50 ok
Muatan sekarang = 849.9999999999999
```

Output:

```
Muatan maksimal = 900.0  
Tambah muatan : 500 Ok  
Tambah muatan : 300 Ok  
Tambah muatan : 150 Gagal  
Tambah muatan : 50 Ok  
Muatan sekarang = 849.9999999999999
```

TUGAS

Tugas 1. Menerapkan konsep enkapsulasi pada kelas Tabungan yang terdapat di Tugas 1. Bab 7. Pengenalan Pemrograman Berbasis Obyek.



Kembangkan kelas Tabungan diatas sehingga memungkinkan pengguna untuk memilih satuan mata uang yang berbeda (USD, AUD, IDR) ketika mengambil atau menyimpan uang. Saldo tabungan disimpan dalam satuan IDR oleh sistem. Beri nama kelas anda dengan nama MultiTabungan.java. Diasumsikan bahwa:

1 AUD = 10.000 IDR

1 USD = 9.000 IDR

Buat kelas baru untuk mengetes kelas MultiTabungan yang anda buat!

SC:

MultiTabungan.java

```
package W7.perbankan;

public class MultiTabungan extends Tabungan {
    public static final int KURS_AUD = 10000;
    public static final int KURS_USD = 9000;

    public MultiTabungan(int initsaldo){
        super(initsaldo); // initsaldo dalam IDR
    }

    private int getKurs(String mataUang){
        switch (mataUang.toUpperCase()) {
            case "IDR": return 1;
            case "AUD": return KURS_AUD;
            case "USD": return KURS_USD;
        }
    }
}
```

```

        default: return -1;
    }
}

public boolean simpanUang(int jumlah, String mataUang){
    int kurs = getKurs(mataUang);
    if (kurs == -1 || jumlah <= 0) {
        return false;
    }
    super.simpanUang(jumlah * kurs);
    return true;
}

public boolean ambilUang(int jumlah, String mataUang){
    int kurs = getKurs(mataUang);
    if (kurs == -1 || jumlah <= 0) {
        return false;
    }
    return super.ambilUang(jumlah * kurs);
}

public double getSaldo(String mataUang){
    int kurs = getKurs(mataUang);
    if (kurs == -1) {
        return -1;
    }
    return (double) getSaldo() / kurs;
}
}

```

TesMultiTabungan.java

```

import W7.perbankan.*;

public class TesMultiTabungan {
    public static void main(String[] args) {
        MultiTabungan tabungan = new MultiTabungan(100000);
        System.out.println("Saldo awal      : " + tabungan.getSaldo() + " IDR");

        tabungan.simpanUang(5, "USD");
        System.out.println("Simpan 5 USD      -> saldo: " + tabungan.getSaldo() + " IDR");

        tabungan.simpanUang(2, "AUD");
        System.out.println("Simpan 2 AUD      -> saldo: " + tabungan.getSaldo() + " IDR");

        tabungan.simpanUang(50000, "IDR");
        System.out.println("Simpan 50000 IDR  -> saldo: " + tabungan.getSaldo() + " IDR");
    }
}

```

```

        boolean status = tabungan.ambilUang(10, "USD");
        System.out.println("Ambil 10 USD      -> " + (status ? "ok" : "gagal") + ", saldo: " +
tabungan.getSaldo() + " IDR");

        status = tabungan.ambilUang(3, "AUD");
        System.out.println("Ambil 3 AUD      -> " + (status ? "ok" : "gagal") + ", saldo: " +
tabungan.getSaldo() + " IDR");

        status = tabungan.ambilUang(100, "AUD");
        System.out.println("Ambil 100 AUD    -> " + (status ? "ok" : "gagal") + ", saldo: " +
tabungan.getSaldo() + " IDR");

        status = tabungan.ambilUang(5, "EUR");
        System.out.println("Ambil 5 EUR      -> " + (status ? "ok" : "gagal (mata uang tidak
dikenal)"));

        System.out.println("Saldo akhir (IDR) : " + tabungan.getSaldo("IDR"));
        System.out.println("Saldo akhir (USD) : " + tabungan.getSaldo("USD"));
        System.out.println("Saldo akhir (AUD) : " + tabungan.getSaldo("AUD"));
    }
}

```

SC:

```

Saldo awal          : 100000 IDR
Simpan 5 USD        -> saldo: 145000 IDR
Simpan 2 AUD        -> saldo: 165000 IDR
Simpan 50000 IDR    -> saldo: 215000 IDR
Ambil 10 USD        -> ok, saldo: 125000 IDR
Ambil 3 AUD         -> ok, saldo: 95000 IDR
Ambil 100 AUD       -> gagal, saldo: 95000 IDR
Ambil 5 EUR         -> gagal (mata uang tidak dikenal)
Saldo akhir (IDR)   : 95000.0
Saldo akhir (USD)   : 10.555555555555555
Saldo akhir (AUD)   : 9.5

```